📋 Planejamento de Projeto e Arquitetura Colaborativa: CatCode (UECE - Extensão II)

> **Projeto:** CatCode — Jogo Educativo de Lógica de Programação com Temática de Proteção Animal
> **Instituição:** Universidade Estadual do Ceará (UECE) — Extensão II / Projeto GABA
> **Motor:** Godot Engine 4.x (GDScript)
> **Documento Base:** Apresentação para a equipe (Alinhamento, Versionamento e Divisão de Tarefas)

---

1. Visão Geral e Nova Identidade (CatCode)

Com base no documento norteador `CatCode.pdf` e nos novos assets disponibilizados:

- **Personagem Jogador:** O jogador programa as ações de um **pequeno Robô assistente da ONG**, responsável por cuidar dos animais acolhidos.
- **Narrativa e Animais Acolhidos:**
  - **Mia:** Gatinha filhote assustada, precisa de abrigo seguro, água e alimentação básica.
  - **Bento:** Cãozinho explorador, percorre áreas com obstáculos do abrigo.
  - **Luna:** Gata adulta em recuperação na enfermaria, precisando de medicamentos e hidratação.
- **Objetivo Pedagógico:** Ensinar algoritmos e lógica (sequenciamento, loops e tomada de decisão) de forma lúdica e sem necessidade de sintaxe tradicional escrita.

---

2. Guia de Versionamento em Equipe: Git & GitHub no Godot 4

Desenvolver jogos em equipe pela primeira vez exige cuidados especiais com o Git, pois **arquivos de cena (`.tscn`) e recursos (`.tres`) são arquivos de texto estruturados** gerados automaticamente pelo Godot. Se duas pessoas editarem a mesma cena ao mesmo tempo, resolver o conflito de merge pode ser complexo.

2.1. O que versionar e o que IGNORAR (`.gitignore`)

Já criamos o arquivo `.gitignore` na raiz do projeto. As regras fundamentais são:

1. **NUNCA versionar a pasta `.godot/`:** Ela contém o cache local de importação, texturas compiladas e configurações de editor da máquina de quem abriu o jogo. Cada membro da equipe gera essa pasta localmente ao abrir o Godot.
2. **Versionar sempre:**
   - Todos os arquivos de script (`.gd`);
   - Todas as cenas (`.tscn`);
   - Arquivo de configuração principal (`project.godot`);
   - Texturas, áudios e fontes originais (`assets/`);
   - Arquivos de UID gerados pelo Godot 4.x (`.uid` e referências de recursos).

2.2. O Godot tem integração com Git?

- O Godot 4 possui suporte a plugins de VCS (Version Control Systems), mas **a melhor prática consolidada na indústria para equipes iniciantes é gerenciar o Git externamente** usando

ferramentas visuais amigáveis:

- **GitHub Desktop (Altamente Recomendado para a Equipe):** Interface limpa, visualização clara das alterações (diffs), troca fácil de branches e resolução guiada de conflitos.
- **VS Code com extensão GitLens / Git Graph:** Excelente para quem já usa o VS Code para editar scripts.
- **Terminal / Git CLI:** Para quem já domina a linha de comando.

2.3. A Regra de Ouro do Godot: Arquitetura Modular por Cenas

Para que várias pessoas trabalhem no jogo ao mesmo tempo sem gerar conflitos no Git:

- **Nunca coloque o jogo inteiro em uma única cena.**
- Cada elemento deve ser sua própria **Cena Isolada (`.tscn`)**:
  - `robot.tscn` (apenas o robô, seu sprite e sua lógica de animação);
  - `hud.tscn` (apenas os botões, paleta e lista de blocos);
  - `start_menu.tscn` (apenas o menu inicial);
  - `level_select.tscn` (apenas a tela de fases);
  - `board.tscn` (o grid e lógica de células);
  - `level_01.tscn`, `level_02.tscn` (apenas o posicionamento das peças daquela fase).
- **Regra de convivência:** Cada membro trabalha na sua branch e edita apenas a cena/script atribuído à sua tarefa.

2.4. Fluxo de Trabalho de Branches (Git Flow Simplificado)

1. `main`: Código de produção estável (sempre funcionando para apresentação aos professores/orientadores).
2. `develop`: Branch de integração onde as funcionalidades prontas são juntadas e testadas.
3. `feature/nome-da-tarefa`: Branches temporárias criadas por cada membro (ex: `feature/menu-iniciar`, `feature/bloco-loop`, `feature/recorte-sprites`).
   - Ao concluir a tarefa, o membro abre um **Pull Request (PR)** para `develop`, a equipe revisa e faz o merge.

---

3. Divisão de Papéis e Tarefas na Equipe

Em desenvolvimento de jogos, a divisão clássica de web ("Back-end vs Front-end") se traduz em **Core / Game Logic** vs **UI / Telas / HUD** vs **Level Design** vs **Arte / Áudio**.

Abaixo está a proposta de divisão para 3 a 5 participantes:

```
┌──────────────────────────────────────────────────────────────────────┐
│                        DIVISÃO DA EQUIPE                             │
├──────────────────────┬──────────────────────┬────────────────────────┤
│ PAPEL                │ RESPONSABILIDADE     │ ARQUIVOS PRINCIPAIS    │
├──────────────────────┼──────────────────────┼────────────────────────┤
│ 1. Core / Gameplay   │ Lógica do Robô,      │ scripts/core/          │
│    (Lógica do Jogo)  │ Grid, Execução       │ scenes/robot/          │
│                      │ dos Comandos         │ command_executor.gd    │
├──────────────────────┼──────────────────────┼────────────────────────┤
```

2. UI / Front-end do Jogo	Telas de Menu, Seleção de Fases, HUD da IDE	scenes/ui/ start_menu.tscn, hud.tscn, level_select.tscn
3. Level Design & Pedagogia	Curadoria dos Mapas, Objetivos, Histórias Animais	scenes/levels/ level_data.gd, fases Mia, Bento e Luna
4. Arte, Sprites & Sonoplastia	Fatiamento da folha de sprites, efeitos sonoros	assets/sprites/ AtlasTexture, sfx/, áudios e ícones

Detalhamento dos Papéis:

👷 **Responsável 1: Core Engine & Interpretador de Blocos ("Back-End do Jogo")**

- Implementar a fila de execução de comandos (Máquina de Estados / Executor).
- Lógica de orientação espacial do Robô (Frente, Virar Esquerda, Virar Direita).
- Detecção de colisão com bordas do grid e obstáculos (árvores, caixas).
- Lógica dos novos blocos: `Coletar`, `Alimentar/Cuidar` e futuramente `Repetir (Loop)`.

🎨 **Responsável 2: Interface, Menus e UX ("Front-End do Jogo")**

- Montar a cena do **Menu Inicial** com base em `menu_iniciar.png`.
- Montar a cena de **Seleção de Fases** com base em `menu_fases_niveis.png`.
- Ajustar a HUD principal com base em `cena_principal.png` (área de blocos montados, paleta de comandos disponíveis, botões Play, Pause, Limpar e Reiniciar).
- Criação dos modais de feedback (Vitória com estrelas / Fracasso pedagógico com dica).

🗺️ **Responsável 3: Level Design, Fases & Narrativa Pedagógica**

- Estruturar os dados das 9 fases divididas nos 3 cenários (Mia, Bento e Luna).
- Configurar posições de largada, posição dos itens (ração, água, remédio), animais e obstáculos em cada fase.
- Escrever os pequenos textos de introdução e diálogo que contextualizam o resgate dos animais no abrigo da ONG.

🖌️ **Responsável 4: Pipeline de Arte, Sprites e Áudio**

- Realizar o recorte dos elementos da folha `sprits.png` (robô nas 4 direções, gato filhote, cãozinho, gato adulto, árvore, grama, itens de cuidado).
- Configurar no Godot como `AtlasTexture` ou salvar PNGs individuais otimizados em `assets/sprites/cutouts/`.
- Selecionar e integrar efeitos sonoros gratuitos (miado, latido, bip do robô, clique de encaixe de blocos, fanfarra de vitória).

---

4. Análise dos Novos Assets Fornecidos

Na pasta `assets/sprites/`, já temos:

1. `menu_iniciar.png`: Layout pronto com Logo CatCode, botão "Iniciar Missão", botão "Configurações" e ilustração temática.
2. `menu_fases_niveis.png`: Mapa de progresso com nós numerados por fase e identificação do animal da missão.
3. `cena_principal.png`: Layout completo da tela de gameplay dividida em:
    - **Lado Esquerdo:** Visão do abrigo em grade com robô, itens, animal e obstáculos.
    - **Lado Direito:** IDE visual com a área de construção do algoritmo e botões de ação.
4. `sprits.png`: Folha completa de sprites contendo personagens, tiles de grama, árvores, tigelas, água e medicamentos.
5. `icone_logo.png`: Logotipo oficial do projeto.

---

5. Cronograma e Entregas por Sprints (Roadmap)

Sugerimos organizar o projeto em **4 Sprints quinzenais**:

🏁 Sprint 1: Fundação, Versionamento e Assets (Semanas 1 e 2)

- [ ] Criação do repositório no GitHub com `.gitignore` oficial do Godot 4.
- [ ] Clonagem do repositório nas máquinas de toda a equipe e teste de commit/push via GitHub Desktop.
- [ ] Fatiamento dos sprites de `sprits.png` e importação organizada no Godot.
- [ ] Refatoração das cenas para arquitetura modular desacoplada (`Robot.tscn`, `HUD.tscn`, `LevelManager.tscn`).

🧩 Sprint 2: Core Loop da Missão da Mia (Semanas 3 e 4)

- [ ] Substituição do gatinho do protótipo pelo sprite do Robô como agente principal.
- [ ] Inclusão da gatinha Mia como objetivo final no mapa.
- [ ] Implementação das ações `Coletar (Ração/Água)` e `Alimentar/Entregar`.
- [ ] Validação dos 3 níveis de dificuldade do Cenário 1 (Alimentação).

🖥️ Sprint 3: Telas de Menu e Navegação Completa (Semanas 5 e 6)

- [ ] Implementação funcional do `menu_iniciar.tscn` (Iniciar -> Seleção de Níveis).
- [ ] Implementação da tela `menu_fases_niveis.tscn` com bloqueio/desbloqueio de fases.
- [ ] Telas de Vitória (Pontuação/Estrelas) e Botão de "Próxima Fase".

🔁 Sprint 4: Bloco de Repetição (Loop), Bento e Luna (Semanas 7 e 8)

- [ ] Implementação do bloco `Repetir (N vezes)` no interpretador.
- [ ] Fases do Bento (caminhos longos com repetições e obstáculos).
- [ ] Fases da Luna (medicação e múltiplos passos lógicos).
- [ ] Adição de áudio e efeitos sonoros finais.
- [ ] Testes de usabilidade e entrega final para o projeto de extensão.

---

6. Ferramentas e Tecnologias Adotadas

Categoria	Ferramenta Recomendada	Justificativa
Engine de Jogos	Godot Engine 4.x (GDScript)	Leve, gratuito, open-source, ideal para grids 2D e arquitetura de cenas.
Controle de Versão	Git + GitHub	Padrão da indústria, rastreabilidade e backup na nuvem.
Cliente Git Visual	GitHub Desktop	Interface simples e intuitiva para evitar acidentes de merge em equipe iniciante.
Gestão de Tarefas	GitHub Projects ou Trello	Quadro Kanban simples: *Backlog*, *A Fazer*, *Em Andamento*, *Concluído*.
Manipulação de Sprites	Aseprite / Krita / Photoshop / GIMP	Para fatiamento e exportação dos sprites da folha `sprits.png`.
Comunicação da Equipe	Discord / WhatsApp + Reuniões Semanais	Alinhamento contínuo sobre quem está editando qual cena.

7. Roteiro Sugerido para a Apresentação de Sexta-feira (30 a 45 min)

1. **Abertura (5 min):** Apresentar a visão do **CatCode**, o objetivo pedagógico (lógica sem código) e a temática de acolhimento animal (GABA/UECE).
2. **Demonstração dos Novos Visuais (10 min):** Mostrar as artes de `menu_iniciar.png`, `menu_fases_niveis.png`, `cena_principal.png` e os personagens (Robô, Mia, Bento, Luna).
3. **Explicação de Como Trabalharemos no Godot e GitHub (10 min):**
    - Mostrar a regra das Cenas Modulares: "cada um no seu arquivo".
    - Demonstrar o GitHub Desktop e a importância do `.gitignore`.
4. **Divisão de Funções (10 min):** Apresentar os 4 pilares e definir quem assume o que.
5. **Dúvidas e Próximos Passos (5 min):** Agendar o primeiro commit conjunto da equipe no repositório.